

Aula 13 - Grafos (Flood Fill)

Um dos algoritmos mais clássicos da OBI relacionados a grafos é a **busca em grid**, muito conhecida por **flood fill**. Basicamente, consiste em uma busca (em profundidade ou em largura) aplicada em um grafo modelado como uma matriz.

Nota: um grafo modelado **COMO** uma matriz, não **COM** uma matriz. Não me refiro a uma DFS/BFS específica de um grafo modelado por matriz de adjacência, mas sim um grafo que essencialmente é uma matriz.

Problema: Existe saída?

Dado um grid quadriculado 5×5, formado apenas por O's e X's, queremos saber se existe um caminho válido entre a primeira célula (`grid[0][0]`) e a última célula (`grid[4][4]`), sendo que só podemos passar por células que contêm O's. Algo como:

$$\text{grid} = \begin{bmatrix} O & O & O & X & X \\ O & X & O & O & O \\ X & O & O & O & X \\ O & O & X & X & X \\ X & O & O & O & O \end{bmatrix} \implies \text{Existe saída!}$$

Notem que, apesar de parecer similar àquele problema da aula de programação dinâmica, nesse, nós não estamos limitados a movimentos para a direita/baixo, somos livres para nos mover em qualquer uma das 4 direções principais (cima, direita, esquerda, baixo), dentro dos limites do grid, claro.

Uma solução possível seria modelar essa matriz inteira em um grafo gigante de 25 vértices e 40 arestas (retirando arestas se um dos vértices for marcado com X) e rodar uma DFS normal partindo do primeiro vértice, mas isso é extremamente chato e ineficiente.

Uma boa forma de resolver esse problema é adaptar o código da DFS/BFS. Ao invés de iterar pelos vizinhos de um determinado vértice para ver quais caminhos possíveis podemos seguir, fazemos uso de uma artimanha empregando combinações de dois pequenos vetores para conseguir as quatro direções:

```

#include <bits/stdc++.h>
using namespace std;

vector<vector<char>> grid = {
    {'0', '0', '0', 'X', 'X'},
    {'0', 'X', '0', '0', '0'},
    {'X', '0', '0', '0', 'X'},
    {'0', '0', 'X', 'X', 'X'},
    {'X', '0', '0', '0', '0'}
};

vector<vector<bool>> vis(5, vector<bool>(5, false));

int dx[4] = {-1, 1, 0, 0};
int dy[4] = {0, 0, -1, 1};

void floodfill(int x, int y){ // ao invés de chamar em um vértice
// chamamos em uma coordenada {x,y} (que na prática define um
"vértice" no grid)

    vis[x][y] = true;

    for(int i = 0; i < 4; i++){

        int nx = x + dx[i]; // coordenada x do vértice vizinho
        int ny = y + dy[i]; // coordenada y do vértice vizinho

        // verifica se o vizinho está dentro dos limites
        if(nx >= 0 && nx < 5 &&
            ny >= 0 && ny < 5){

            // verifica se podemos visitar o vizinho

```

```

        if(grid[nx][ny] == '0' &&
           !vis[nx][ny]){

            floodfill(nx, ny);

        }
    }
}

int main(){

    floodfill(0,0);

    if(vis[4][4]) cout << "Existe saída!";
    else cout << "Não existe saída :(";

}

```

A grande ideia que simplifica muito o código é o cálculo das coordenadas dos possíveis vizinhos combinando os dois vetores `dx` e `dy` com o loop:

```

for(int i = 0; i<4; ++i){
    int nx = x + dx[i];
    int ny = y + dy[i];
    //...
}

```

Na primeira iteração, temos $nx = x + (-1)$ e $ny = y + 0$, isso é a mesma coisa que “o vértice à esquerda do vértice atual”. Na segunda, temos $nx = x + 1$ e $ny = y + 0$, que é a mesma coisa que “o vértice à direita do vértice atual”.

O mesmo se repete para verificarmos o vértice acima e abaixo do vértice atual.

Essa é a implementação recursiva de uma flood fill, que simula uma **dfs** (por seguir o mesmo comportamento recursivo). Poderíamos implementar a versão iterativa com o uso de uma pilha que também tem o mesmo comportamento da dfs comum:

```

#include <bits/stdc++.h>
using namespace std;

int dx[4] = {-1, 1, 0, 0};
int dy[4] = {0, 0, -1, 1};

void floodfillDFS(vector<vector<char>>& grid, vector<vector<bool>>& vis,
    int sx, int sy){

    stack<pair<int,int>> st;

    st.push({sx, sy});
    vis[sx][sy] = true;

    while(!st.empty()){

        pair<int,int> p = st.top();
        int x = p.first, y = p.second;
        st.pop();

        for(int i = 0; i < 4; i++){

            int nx = x + dx[i];
            int ny = y + dy[i];

            bool dentro = nx >= 0 && nx < n && y >= 0 && ny <
m;

            if(dentro && !vis[nx][ny] && grid[nx][ny] == '0')
            {
                vis[nx][ny] = true;
                st.push({nx, ny});
            }
        }
    }
}

```

```
}  
}
```

E, da mesma forma como na dfs comum, se substituirmos a pilha por uma fila, temos uma flood fill com comportamento de bfs.